

Universidad de Guadalajara
Sistema de Educación Superior
Centro Universitario de los Valles



Docente: Adolfo Castillo Chavarín

Alumno: Ulises Sánchez Camarena

Materia: Desarrollo de bases de datos para web

Código: 218408236

Actividad: U1 - A4 Introducción a la programación web con Python -
MySQL- Flask

Fecha: 24/09/2026

1. Introducción

Esta práctica consistió en desarrollar una aplicación web tipo CRUD (Crear, Leer, Actualizar, Eliminar) para gestionar un catálogo de videojuegos, usando Python con el framework Flask para la parte web y MySQL (que yo use a través de XAMPP) como base de datos. El propósito fue aplicar de forma práctica los conceptos de bases de datos relacionales y desarrollo web con Python: diseñar una tabla, conectar Python a MySQL con la librería pymysql, y construir rutas, controladores y plantillas HTML que permitieran administrar los registros desde el navegador. El objetivo final fue tener una aplicación funcional donde se pudieran agregar, listar, editar y eliminar juegos.

2. Desarrollo

Creación de la base de datos

El primer paso fue levantar el servicio de MySQL desde el Panel de Control de XAMPP e ingresar al cliente de línea de comandos con `mysql -u root -p`. Desde ahí se creó la base de datos juegos y, dentro de ella, la tabla juegos con los campos id (llave primaria autoincremental), nombre, descripcion y precio. Se verificó la estructura con `SHOW TABLES` y `DESCRIBE juegos`, y se insertaron algunos registros de prueba.

```
Administrador: XAMPP for Windows - mysql -u root -p
1 row in set (0.001 sec)

MariaDB [juegos]> describe juegos
-> describe juegos;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'describe juegos' at line 2
MariaDB [juegos]> describe juegos;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id     | bigint(20) unsigned | NO | PRI | NULL | auto_increment |
| nombre | varchar(255)         | NO |     | NULL |                 |
| descripcion | varchar(255)       | NO |     | NULL |                 |
| precio | decimal(9,2)         | NO |     | NULL |                 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.026 sec)

MariaDB [juegos]> select * from juegos;
Empty set (0.050 sec)

MariaDB [juegos]> select * from juegos;
+----+-----+-----+-----+
| id | nombre      | descripcion      | precio |
+----+-----+-----+-----+
| 2  | Mario Kart 8 | Juego de Carreras | 1500.00 |
| 3  | Mario Mender | Juego de plataformas | 1600.00 |
| 4  | Mario Kart World | Juego bien chingon de carreras | 2000.00 |
+----+-----+-----+-----+
3 rows in set (0.001 sec)

MariaDB [juegos]>
```

Conexión desde Python

Se creó un archivo `bd.py` con una función `obtener_conexion()` que usa `pymysql.connect()` para conectarse a la base de datos, indicando `host`, `usuario`, `contraseña` y el `nombre` de la base. Esta función es la que reutilizan el resto de los archivos del proyecto para no repetir los datos de conexión.

```
main.py  bd.py X
mini_sistema_de_videojuegos > bd.py > obtener_conexion
1  import pymysql
2
3
4  def obtener_conexion():
5      return pymysql.connect(host='localhost',
6                             user='root',
7                             password='admin123',
8                             db='juegos')
```

Controlador y lógica de negocio

En `controlador_juegos.py` se definieron las funciones que ejecutan las consultas SQL: `obtener_juegos`, `insertar_juego`, `obtener_juego_por_id`, `actualizar_juego` y `eliminar_juego`. Cada una abre una conexión, ejecuta su consulta correspondiente y cierra la conexión.

```
main.py  controlador_juegos.py X
mini_sistema_de_videojuegos > controlador_juegos.py > eliminar_juego
1  from bd import obtener_conexion
2
3
4  def insertar_juego(nombre, descripcion, precio):
5      conexion = obtener_conexion()
6      with conexion.cursor() as cursor:
7          cursor.execute("INSERT INTO juegos(nombre, descripcion, precio) VALUES (%s, %s, %s)",
8                          (nombre, descripcion, precio))
9      conexion.commit()
10     conexion.close()
11
12
13  def obtener_juegos():
14     conexion = obtener_conexion()
15     juegos = []
16     with conexion.cursor() as cursor:
17         cursor.execute("SELECT id, nombre, descripcion, precio FROM juegos")
18         juegos = cursor.fetchall()
19     conexion.close()
20     return juegos
21
22
23  def eliminar_juego(id):
24     conexion = obtener_conexion()
25     with conexion.cursor() as cursor:
26         cursor.execute("DELETE FROM juegos WHERE id = %s", (id,))
27     conexion.commit()
28     conexion.close()
29
30
31  def obtener_juego_por_id(id):
32     conexion = obtener_conexion()
33     juego = None
34     with conexion.cursor() as cursor:
35         cursor.execute("SELECT id, nombre, descripcion, precio FROM juegos WHERE id = %s", (id,))
36     return juego
```

```
main.py  controlador_juegos.py X
mini_sistema_de_videojuegos > controlador_juegos.py > eliminar_juego
23 def eliminar_juego(id):
28     conexion.close()
29
30
31 def obtener_juego_por_id(id):
32     conexion = obtener_conexion()
33     juego = None
34     with conexion.cursor() as cursor:
35         cursor.execute(
36             "SELECT id, nombre, descripcion, precio FROM juegos WHERE id = %s", (id,))
37         juego = cursor.fetchone()
38     conexion.close()
39     return juego
40
41
42 def actualizar_juego(nombre, descripcion, precio, id):
43     conexion = obtener_conexion()
44     with conexion.cursor() as cursor:
45         cursor.execute("UPDATE juegos SET nombre = %s, descripcion = %s, precio = %s WHERE id = %s",
46             (nombre, descripcion, precio, id))
47     conexion.commit()
48     conexion.close()
49
```

Aplicación Flask

En main.py se definió la aplicación Flask con sus rutas:

- /juegos: muestra la lista de juegos.
- /agregar_juego y /guardar_juego: muestran el formulario de alta y procesan el guardado.
- /formulario_editar_juego/<id> y /actualizar_juego: muestran el formulario de edición con los datos precargados y procesan la actualización.
- /eliminar_juego: elimina un registro.

Cada ruta de "acción" (guardar, actualizar, eliminar) usa el método POST para recibir los datos del formulario mediante request.form, y al terminar redirige de vuelta a /juegos.

```

main.py X controlador_juegos.py
mini_sistema_de_videojuegos > main.py >...
1 from flask import Flask, render_template, request, redirect
2 import controlador_juegos # si lo dejas como juegos.py: import juegos as controlador_juegos
3
4 app = Flask(__name__)
5
6 @app.route("/")
7 @app.route("/juegos")
8 def juegos():
9     juegos = controlador_juegos.obtener_juegos()
10    return render_template("juegos.html", juegos=juegos)
11
12 @app.route("/agregar_juego")
13 def formulario_agregar_juego():
14     return render_template("agregar_juego.html")
15
16 @app.route("/guardar_juego", methods=["POST"])
17 def guardar_juego():
18     nombre = request.form["nombre"]
19     descripcion = request.form["descripcion"]
20     precio = request.form["precio"]
21     controlador_juegos.insertar_juego(nombre, descripcion, precio)
22     return redirect("/juegos")
23
24 @app.route("/eliminar_juego", methods=["POST"])
25 def eliminar_juego():
26     controlador_juegos.eliminar_juego(request.form["id"])
27     return redirect("/juegos")
28
29 @app.route("/formulario_editar_juego/<int:id>")
30 def editar_juego(id):
31     juego = controlador_juegos.obtener_juego_por_id(id)
32     return render_template("editar_juego.html", juego=juego)
33
34 @app.route("/actualizar_juego", methods=["POST"])
35 def actualizar_juego():
36     id = request.form["id"]
37     nombre = request.form["nombre"]

```

Plantillas HTML

Se usó herencia de plantillas de Jinja: una plantilla base maestra.html con el <head> y el navbar, de la cual heredan ({% extends %}) las demás páginas: juegos.html (la tabla), agregar_juego.html y editar_juego.html (los formularios). Esto evitó repetir el HTML común en cada archivo.

```

main.py X maestra.html juegos.html editar_juego.html agregar_juego.html
mini_sistema_de_videojuegos > templates > maestra.html > html > body > navbar.is-warning > div.navbar-menu > div.navbar-start > a.navbar-item
1 <!DOCTYPE html>
2 <html lang="es">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta http-equiv="X-UA-Compatible" content="IE=edge">
7     <meta name="viewport" content="width=device-width, initial-scale=1.0">
8     <title>CRUD MySQL con Flask - {% block titulo %}{% endblock %}</title>
9     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bulma@0.9.2/css/bulma.min.css">
10 </head>
11
12 <body>
13     <nav class="navbar is-warning" role="navigation" aria-label="main navigation">
14         <div class="navbar-brand">
15             <a href="https://parzibyte.me/blog" class="navbar-item">
16                 
19             </a>
20             <button class="navbar-burger is-warning button" aria-label="menu" aria-expanded="false"
21                 data-target="#navbarBasicExample">
22                 <span aria-hidden="true"></span>
23                 <span aria-hidden="true"></span>
24                 <span aria-hidden="true"></span>
25             </button>
26         </div>
27         <div class="navbar-menu">
28             <div class="navbar-start">
29                 <a class="navbar-item" href="{url_for('juegos')}">Juegos</a>
30             </div>
31             <div class="navbar-end">
32                 <div class="navbar-item">
33                     <div class="buttons">
34                         <a href="https://parzibyte.me/contacto" class="button is-primary">
35                             <strong>Ayuda y soporte</strong>
36                         </a>

```

```

main.py X  maestra.html  juegos.html X  editar_juego.html  agregar_juego.html
mini_sistema_de_videojuegos > templates > juegos.html > ...
1  {% extends "maestra.html" %}
2  {% block titulo %} Juegos {% endblock %}
3  {% block contenido %}
4
5  <section class="section">
6      <div class="columns">
7          <div class="column">
8              <h3 class="is-size-3">Juegos</h3>
9              <a href="{{url_for('formulario_agregar_juego')}}" class="button is-success">Agregar</a>
10             <table class="table">
11                 <thead>
12                     <tr>
13                         <th>Nombre</th>
14                         <th>Descripción</th>
15                         <th>Precio</th>
16                         <th>Editar</th>
17                         <th>Eliminar</th>
18                     </tr>
19                 </thead>
20                 <tbody>
21                     {% for juego in juegos %}
22                     <tr>
23                         <td>{{juego[1]}}</td>
24                         <td>{{juego[2]}}</td>
25                         <td>{{juego[3]}}</td>
26                         <td>
27                             {# Pasar el id (lo que hay en juego[0] #)
28                             <a href="{{url_for('editar_juego', id=juego[0])}}" class="button is-info">Editar</a>
29                         </td>
30                         <td>
31                             <form action="{{url_for('eliminar_juego')}}" method="POST">
32                                 <input type="hidden" name="id" value="{{juego[0]}}">
33                                 <button class="button is-danger">Eliminar</button>
34                             </form>
35                         </td>
36                     </tr>
37                 </tbody>
38             </table>
39         </div>
40     </div>
41 </section>
42 {% endblock %}

```

```

main.py X  maestra.html  juegos.html  editar_juego.html X  agregar_juego.html
mini_sistema_de_videojuegos > templates > editar_juego.html > ...
1  {% extends "maestra.html" %}
2  {% block titulo %}
3  Editar juego
4  {% endblock %}
5  {% block contenido %}
6  <section class="section">
7      <div class="columns">
8          <div class="column">
9              <h3 class="is-size-3">Editar juego</h3>
10             <form action="{{url_for('actualizar_juego')}}" method="post">
11                 {# Enviar el id en un campo oculto #}
12                 <input type="hidden" name="id" value="{{juego[0]}}">
13                 <label for="nombre">Nombre:</label>
14                 <input value="{{juego[1]}}" required type="text" class="input" placeholder="Nombre" name="nombre" id="nombre">
15                 <label for="descripcion">Descripción</label>
16                 <input value="{{juego[2]}}" required type="text" class="input" placeholder="Descripción" name="descripcion" id="descripcion">
17                 <label for="precio">Precio</label>
18                 <input value="{{juego[3]}}" required type="number" class="input" placeholder="Precio" name="precio" id="precio">
19                 <button type="submit" class="button is-success mt-2">Guardar</button>
20                 <a href="{{url_for('juegos')}}" class="button is-primary mt-2">Volver</a>
21             </form>
22         </div>
23     </div>
24 </section>
25 {% endblock %}
26

```

```

mini_sistema_de_videojuegos > templates > agregar_juego.html > section.section
1  {% extends "maestra.html" %}
2  {% block titulo %}
3  Agregar juego
4  {% endblock %}
5  {% block contenido %}
6  <section class="section">
7      <div class="columns">
8          <div class="column">
9              <h3 class="is-size-3">Agregar juego</h3>
10             <form action="{{url_for('guardar_juego')}}" method="post">
11                 <label for="nombre">Nombre:</label>
12                 <input required type="text" class="input" placeholder="Nombre" name="nombre" id="nombre">
13                 <label for="descripcion">Descripción</label>
14                 <input required type="text" class="input" placeholder="Descripción" name="descripcion" id="descripcion">
15                 <label for="precio">Precio</label>
16                 <input required type="number" class="input" placeholder="Precio" name="precio" id="precio">
17                 <button type="submit" class="button is-success mt-2">Guardar</button>
18                 <a href="{{url_for('juegos')}}" class="button is-primary mt-2">Volver</a>
19             </form>
20         </div>
21     </div>
22 </section>
23 {% endblock %}
24
25

```

Problemas encontrados y cómo se resolvieron

- Advertencia de Pylance ("Import pymysql could not be resolved"): VS Code no tenía seleccionado el intérprete del entorno virtual donde estaba instalado pymysql. Se resolvió seleccionando el intérprete correcto con Python: Select Interpreter.
- Bloqueo de PowerShell al activar el entorno virtual: PowerShell impedía ejecutar Activate.ps1 por política de ejecución de scripts. Se resolvió con Set-ExecutionPolicy -Scope CurrentUser RemoteSigned.
- La aplicación "no hacía nada" al ejecutarla: los archivos tenían cambios sin guardar (se notaba por el punto en las pestañas de VS Code), por lo que Python ejecutaba una versión desactualizada o incompleta. Se resolvió guardando todos los archivos antes de correr python main.py.
- Error "can't open file main.py": ocurría porque el comando se ejecutaba desde la carpeta Proyectos_Tareas en lugar de la subcarpeta del proyecto. Se resolvió ubicándose primero en la carpeta correcta con cd.

3. Lecciones Aprendidas

Al implementar la práctica quedó más clara la diferencia entre las distintas capas de una aplicación web: la base de datos (MySQL), la lógica de acceso a datos (el controlador), las rutas que exponen esa lógica al navegador (Flask) y la presentación (las plantillas HTML). En el material esto se ve por separado, pero al construirlo se entendió mejor cómo se comunican entre sí, por ejemplo, cómo un clic en un botón de la tabla termina ejecutando un UPDATE en MySQL.

También se reforzó el concepto de herencia de plantillas con `{% extends %}`, que no estaba tan claro hasta verlo funcionando: un solo cambio en la plantilla base se refleja en todas las páginas que heredan de ella.

Las habilidades que más se fortalecieron fueron la conexión de Python a bases de datos y la construcción de rutas Flask. Como área de oportunidad, falta profundizar en el manejo de entornos virtuales y en la depuración de errores de configuración (permisos, rutas, intérpretes), que consumieron más tiempo que la programación en sí.

4. Conclusiones

Se logró construir una aplicación funcional que permite administrar un catálogo de videojuegos completo con las cuatro operaciones básicas: agregar, listar, editar y eliminar, conectando Python con una base de datos MySQL mediante Flask.

Esta práctica deja una base reutilizable para futuros proyectos: la misma estructura (conexión, controlador, rutas, plantillas) se puede aplicar a otro tipo de catálogos o sistemas con pocos cambios, y el manejo de entornos virtuales y dependencias es una habilidad que se necesitará en cualquier proyecto de Python que crezca más allá de un solo archivo.

